# Algorithms: Design and Analysis, Part II

Mihir Rao

## Contents

**1 Motivating Applications** **2**
1.1 Sequence alignment . . . . . 2
1.2 Distributed shortest-path routing . . . . . 2

# Lecture 1: Motivating Applications

## 1.1 Sequence alignment

Sequence alignment compares strings by placing their symbols in columns, with gaps allowed to represent insertions or deletions. For genomic sequences over $\{A, C, G, T\}$, one alignment of `AGGGCT` and `AGGCA` is

$$\begin{array}{cccccc} A & G & G & G & C & T \\ A & G & G & - & C & A \end{array}$$

It has one gap and one mismatch. Given nonnegative penalties for gaps and symbol pairs, the sequence alignment problem asks for an alignment of minimum total penalty. A low score indicates similarity under the chosen penalty model; its meaning depends on how those penalties are selected.

The definition is useful only if the best alignment can be found. Exhaustively trying alignments grows exponentially: even for two strings of length 500, there are at least $2^{500}$ distinct alignments. Dynamic programming exploits repeated subproblems to find an optimum efficiently. The central modeling step is to turn an informal idea of similarity into a precise objective whose solutions can be computed.

## 1.2 Distributed shortest-path routing

A communication network can be modeled by a graph: vertices represent hosts or routers, and edges represent direct connections. Routing from one gateway to another requires choosing a path, for example one with the fewest hops or the least total edge cost. With nonnegative lengths, Dijkstra's algorithm solves the shortest-path problem when the graph is available to the computation.

A router may not know the entire network. A distributed formulation instead uses local communication: a vertex can update its estimate of the cost to a destination from the estimates advertised by its neighbors and the costs of its incident links. The Bellman–Ford approach builds on this local recurrence and also accommodates negative edge costs when shortest paths are well-defined. The lesson is that the same optimization problem can call for a different algorithm when information is distributed.

**Source.** Tim Roughgarden, Algorithms: Design and Analysis, Part II, Part 1 slides.